

Network Security Mini Project PKI, Firewall, and TLS Analysis

**A practical implementation of Public Key Infrastructure (PKI),
HTTPS configuration, firewall setup using iptables, and TLS
handshake analysis using Wireshark.**

CIS-453 Network Security

Prepared by: Muhannad Alamri

Instructions	1
Task-1 (Write the appropriate SEED Commands to do the following)	2
1- Generating the CA Private Key	2
2- Generating the Server Private Key.....	2
3- Creating the Certificate Signing Request (CSR).....	2
4- Signing the Server Certificate Using the CA.....	2
5- Displaying the Final Certificate Details	2
Step-1.....	2
step-2.....	2
step-3.....	2
Task-2(Building a simple Firewall on the client machine and answer every question below)	2
1- Setting Default Firewall Policies to DROP	2
2- Allowing HTTP and HTTPS Forwarding Traffic	2
3- Blocking a Malicious IP Address.....	2
4- Allowing Incoming HTTP and HTTPS Requests.....	2
5- Enabling Outbound DNS Traffic	2
Task-3(Establish a handshake session with any trusted website and show the following)	2
1. What is the IP address of the website >> use nslookup in the command terminal.....	2
2- Client Hello message and show a cipher suite supported by your browser	2
3- What is the compression method?.....	2
4- What is the session ID?.....	2
5- What is a random number?	2
6- Inspect Server Hello, what did the server choose in the cipher suite?	2
In the Server Hello message, the server selects one cipher suite from the list offered by the client to secure the TLS session. The selected cipher suite in our captured packet is	2
TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256.....	2
This cipher suite provides strong encryption and authentication using ECDHE and AES-GCM, helping ensure the confidentiality and integrity of the connection.	2
7- Is the session ID the same from Server Hello What does this refer to ?	2
8- Is the random number the same from Server Hello? Why ?	2
9- Is there a certificate packet between KCU and your browser? Why ?	2
10- Inspect Change cipher spec, do you see the contents of the message? If not, why ?.....	2
11- Open any other website (https) and show the certificate of the website and explain its contents captured by Wireshark.	2

453 CIS-3 NETWORK SECURITY

Mini-Project

Instructions:

You must adhere to the instructions of the project:

1. Answer all the questions as required
2. The answer is by capturing the screen and attaching it to each question.
3. Each group consists of three students only.
4. Delay in project submission will be marked as zero.
5. Copying the answer between groups will be a case of cheating and the project mark will be deducted.

1. Write the appropriate SEED Commands to do the following: (2 marks)

1. Implement PKI approach to issue a digital certificate with name of **Student1Student2Student3.crt** and do authentication on SEEDPKILab2025.com certificate. MAKE SURE THAT YOU USE YOUR KKU EMAILADDRESS IN THE CERTIFICATE.

Hint Remember the steps:

- Become a certificate authority CA.
- Then generate a self-signed certificate to be used as CA.
- Generate public/private key for the server.
- Generate a Certificate Signing Request (CSR) for the server.
- Generate CA to sign the requested certificate.
- Show the certificate.

2. Building a simple Firewall on the client machine and answer every question below:(2 Marks)

1- Change the Default Chain Policies to Drop

- Why is it advisable to change the default chain policies to "Drop" in a firewall configuration?
- What potential security risks are mitigated by setting default policies to "Drop" instead of "Accept"?

2- Allow forwarding of TCP traffic on IP interface 10.0.1.2 (client) port 80 (HTTP) and port 443 (HTTPS) to go to 192.168.23.10

- Explain the significance of allowing TCP traffic on ports 80 and 443 for a client machine.
- How does this rule contribute to securing web traffic for the client?

3- block a specific ip-address from reaching the organization.

- What scenarios might warrant blocking a specific IP address in a firewall configuration?

- How can blocking a specific IP address contribute to network security?

4- Allow Incoming HTTP and HTTPS

- Why is it important to explicitly allow incoming HTTP and HTTPS traffic in a firewall configuration?
- In what situations might blocking incoming HTTP and HTTPS be necessary for security reasons?

5- Allow outbound DNS.

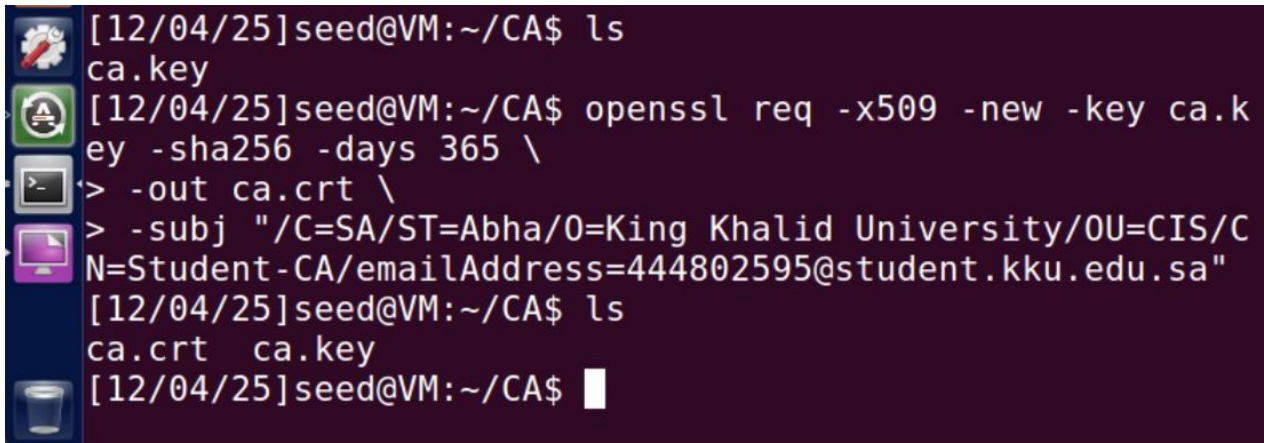
- Discuss the reasons for allowing outbound DNS traffic in a firewall configuration.
- How might blocking outbound DNS impact the normal operation of the client machine?

3. Establish a handshake session with any trusted website and show the following : (3 Marks)

1. What is the IP address of the website >> use nslookup in the command terminal .
Then from Wireshark, inspect the following:
2. Client Hello message and show a cipher suite supported by your browser .
3. What is the compression method?
4. What is the session ID?
5. What is a random number?
6. Inspect Server Hello, what did the server choose in the cipher suite?
7. Is the session ID the same from Server Hello What does this refer to ?
8. Is the Random number the same from Server Hello? Why ?
9. Is there a certificate packet between KKU and your browser? Why ?
10. Inspect Change cipher spec, do you see the contents of the message? If not, why ?
11. Open any other website (https) and show the certificate of the website and explain its contents captured by Wireshark.

Task-1 (Write the appropriate SEED Commands to do the following):

1- Generating the CA Private Key



```
[12/04/25]seed@VM:~/CA$ ls
ca.key
[12/04/25]seed@VM:~/CA$ openssl req -x509 -new -key ca.k
ey -sha256 -days 365 \
> -out ca.crt \
> -subj "/C=SA/ST=Abha/O=King Khalid University/OU=CIS/C
N=Student-CA/emailAddress=444802595@student.kku.edu.sa"
[12/04/25]seed@VM:~/CA$ ls
ca.crt  ca.key
[12/04/25]seed@VM:~/CA$
```

In this step, we create the Certificate Authority (CA) private key and generate a self-signed certificate for the CA. This certificate will later be used to sign and verify the server's certificate, acting as the trusted authority.

2- Generating the Server Private Key



```
[12/04/25]seed@VM:~/CA$ mkdir -p ~/server
[12/04/25]seed@VM:~/CA$ cd ~/server
[12/04/25]seed@VM:~/server$ openssl genrsa -out server.k
ey 2048
Generating RSA private key, 2048 bit long modulus
.....+++
.....+++
e is 65537 (0x10001)
[12/04/25]seed@VM:~/server$ ls
server.key
[12/04/25]seed@VM:~/server$
```

Here, we generate the private key for the server. This key will be used to secure the communication between the server and clients when using HTTPS, ensuring encryption and confidentiality.

3- Creating the Certificate Signing Request (CSR)

```
.....+++
e is 65537 (0x10001)
[12/04/25]seed@VM:~/server$ ls
server.key
[12/04/25]seed@VM:~/server$ openssl req -new -key server
.key -out server.csr \
> -subj "/C=SA/ST=Abha/O=King Khalid University/OU=CIS/CN=SEEDPKILab2025.com/emailAddress=444802595@student.kku.edu.sa"
[12/04/25]seed@VM:~/server$ ls
server.csr  server.key
[12/04/25]seed@VM:~/server$
```

In this step, we generate a CSR for the server using its private key. The CSR includes server identity information (such as domain name and organization) and will be sent to the CA to request a signed certificate.

4- Signing the Server Certificate Using the CA

```
[12/04/25]seed@VM:~/server$ cd ~/CA
[12/04/25]seed@VM:~/CA$ openssl x509 -req -in ~/server/s
erver.csr -CA ca.crt -CAkey ca.key \
> -CAcreateserial -out ~/server/MuhannadOsamaTurkiAbdull
ah.crt -days 365 -sha256
Signature ok
subject=/C=SA/ST=Abha/O=King Khalid University/OU=CIS/CN
=SEEDPKILab2025.com/emailAddress=444802595@student.kku.e
du.sa
Getting CA Private Key
[12/04/25]seed@VM:~/CA$
```

Here, the CA uses its private key to sign the server's CSR and generate a valid server certificate. This allows the server to be trusted by clients when establishing a secure HTTPS connection.

```
SEEDUbuntu-16.04 [Running] - Oracle VirtualBox
Terminal
> -CAcreateserial -out ~/server/MuhammadOsamaTurkiAbdullah.crt -days 365 -sha256
Signature ok
subject=/C=SA/ST=Abha/O=King Khalid University/OU=CIS/CN=SEEDPKILab2025.com/emailAddress=444802595@student.kku.edu.sa
Getting CA Private Key
[12/04/25]seed@VM:~/CA$ cd ~/server
[12/04/25]seed@VM:~/server$ openssl x509 -in MuhammadOsamaTurkiAbdullah.crt -text -noout
Certificate:
    Data:
        Version: 1 (0x0)
        Serial Number:
            be:93:37:f3:21:92:4f:73
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=SA, ST=Abha, O=King Khalid University, OU=CIS, CN=Student-CA/emailAddress=444802595@student.kku.edu.sa
        Validity
            Not Before: Dec  4 23:31:44 2025 GMT
            Not After : Dec  4 23:31:44 2026 GMT
```

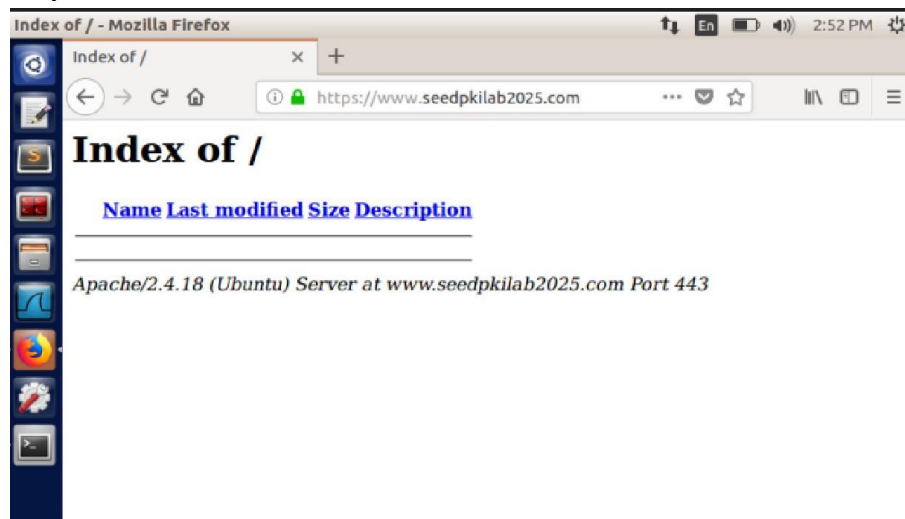
After configuring Apache with the generated SSL certificate, the website was accessed using HTTPS on port 443. The browser successfully loaded the page, confirming that the certificate is correctly installed and that the connection between the browser and the server is encrypted using TLS.

5- Displaying the Final Certificate Details

Step-1

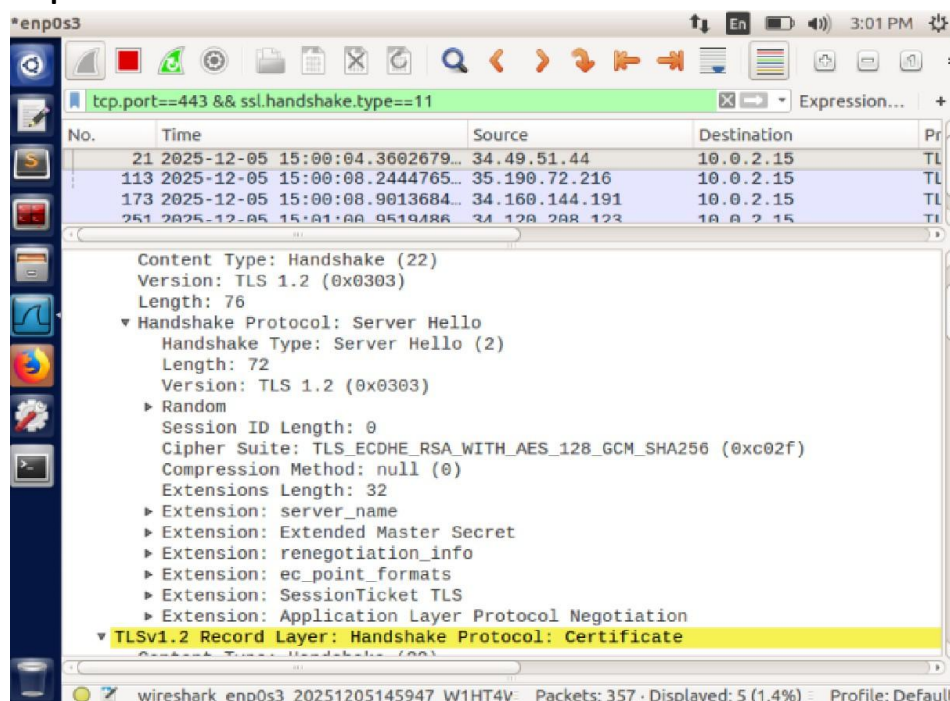
```
Terminal
[12/04/25]seed@VM:~/server$ openssl x509 -in MuhammadOsamaTurkiAbdullah.crt -text -noout
Certificate:
    Data:
        Version: 1 (0x0)
        Serial Number:
            be:93:37:f3:21:92:4f:73
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=SA, ST=Abha, O=King Khalid University, OU=CIS, CN=Student-CA/emailAddress=444802595@student.kku.edu.sa
        Validity
            Not Before: Dec  4 23:31:44 2025 GMT
            Not After : Dec  4 23:31:44 2026 GMT
        Subject: C=SA, ST=Abha, O=King Khalid University, OU=CIS, CN=SEEDPKILab2025.com/emailAddress=444802595@student.kku.edu.sa
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
                00:c9:35:e5:72:36:0b:39:c9:9f:3e:00:
```

step-2



After configuring Apache with the generated SSL certificate, the website was accessed using HTTPS on port 443. The browser successfully loaded the page, confirming that the certificate is correctly installed and that the connection between the browser and the server is encrypted using TLS.

step-3

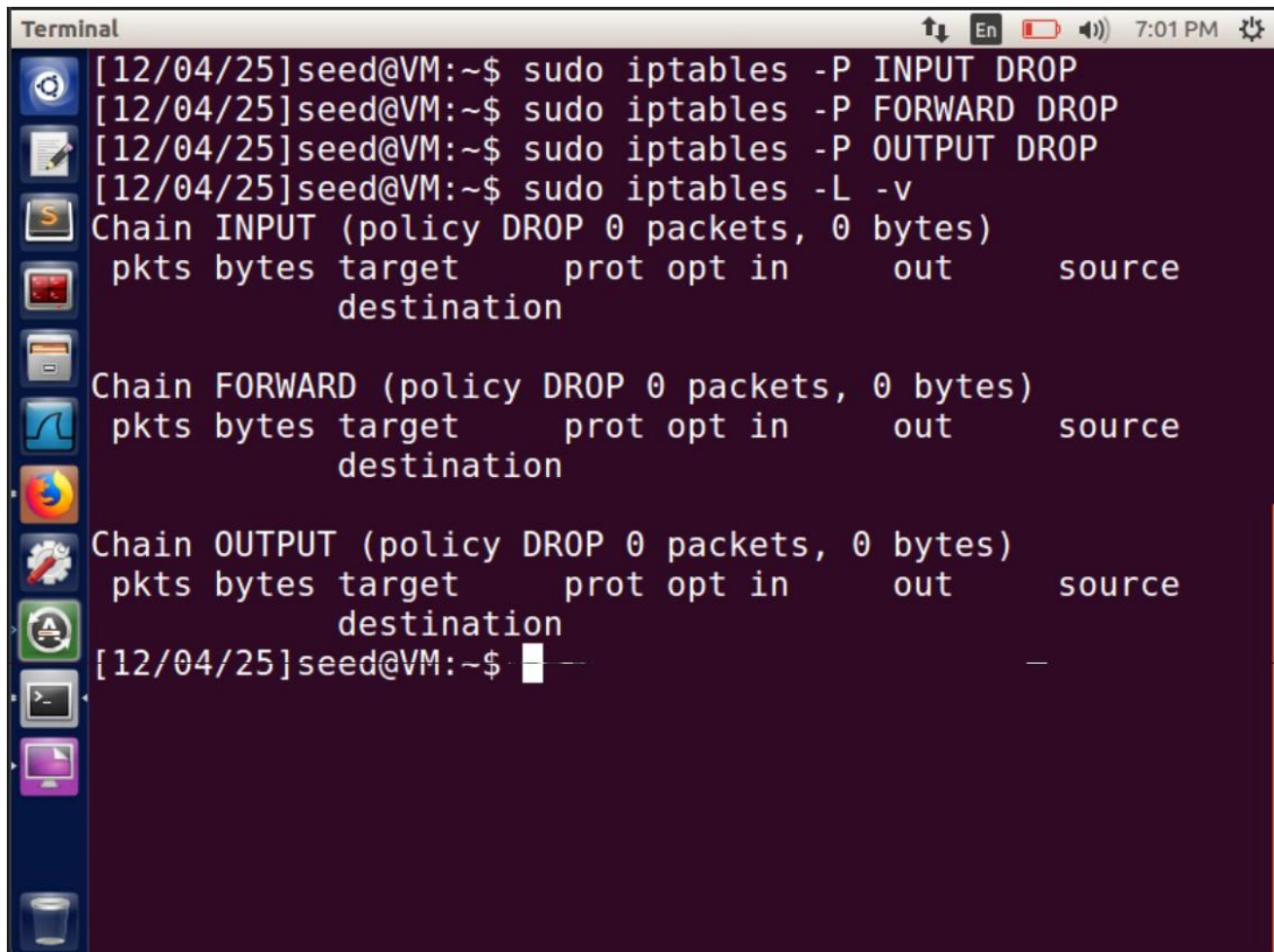


Wireshark was used to analyze the TLS handshake process. By applying a filter to capture handshake messages, the "Certificate" message was detected. This message shows that the server sends its SSL certificate to the client during the TLS handshake, proving that encryption and identity verification are taking place.

Task-2(Building a simple Firewall on the client machine and answer every question below):

This task aims to build a secure firewall using iptables by applying a default-deny approach and selectively allowing only necessary traffic such as web browsing and DNS while blocking unauthorized access.

1- Setting Default Firewall Policies to DROP



```
Terminal
[12/04/25]seed@VM:~$ sudo iptables -P INPUT DROP
[12/04/25]seed@VM:~$ sudo iptables -P FORWARD DROP
[12/04/25]seed@VM:~$ sudo iptables -P OUTPUT DROP
[12/04/25]seed@VM:~$ sudo iptables -L -v
Chain INPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target      prot opt in      out     source
      destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target      prot opt in      out     source
      destination

Chain OUTPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target      prot opt in      out     source
      destination
[12/04/25]seed@VM:~$
```

In this step, the default firewall policies for all chains (INPUT, FORWARD, and OUTPUT) are set to DROP. This means that any network traffic will be blocked by default unless a specific rule is added to allow it. Setting DROP as the default policy helps protect the client machine by preventing unauthorized access and stopping unexpected connections.

2- Allowing HTTP and HTTPS Forwarding Traffic

```
[12/04/25]seed@VM:~$ sudo iptables -A FORWARD -p tcp -d 192.168.23.10 --dport 80 -j ACCEPT
[12/04/25]seed@VM:~$ sudo iptables -A FORWARD -p tcp -d 192.168.23.10 --dport 443 -j ACCEPT
[12/04/25]seed@VM:~$ sudo iptables -L -v
Chain INPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target      prot opt in      out     source
      destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target      prot opt in      out     source
      destination
      0      0 ACCEPT     tcp  --  any    any    anywhere
      192.168.23.10      tcp dpt:http
      0      0 ACCEPT     tcp  --  any    any    anywhere
      192.168.23.10      tcp dpt:https

Chain OUTPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target      prot opt in      out     source
      destination
[12/04/25]seed@VM:~$
```

In this step, rules are added to allow forwarding of TCP traffic on ports 80 (HTTP) and 443 (HTTPS) to the internal server at 192.168.23.10. These ports are used for secure and regular web browsing. By allowing only this specific traffic, the firewall ensures that the client machine can access web services securely while still blocking other unwanted traffic.

3- Blocking a Malicious IP Address

```
[12/04/25]seed@VM:~$ sudo iptables -A INPUT -s 10.0.1.10
0 -j DROP
[12/04/25]seed@VM:~$ sudo iptables -L -v
Chain INPUT (policy DROP 0 packets, 0 bytes)
  pkts bytes target     prot opt in       out     source
      0      0 DROP      all  --  any     any     10.0.1.10
0      anywhere
Chain FORWARD (policy DROP 0 packets, 0 bytes)
  pkts bytes target     prot opt in       out     source
      0      0 ACCEPT    tcp  --  any     any     anywhere
      0      0          192.168.23.10 tcp dpt:http
      0      0 ACCEPT    tcp  --  any     any     anywhere
      0      0          192.168.23.10 tcp dpt:https
Chain OUTPUT (policy DROP 0 packets, 0 bytes)
  pkts bytes target     prot opt in       out     source
      0      0
```

In this step, a malicious IP address (10.0.1.10) is blocked using the firewall rule. The rule prevents any incoming traffic from this specific IP from reaching the client machine by dropping all packets from that source.

This is important for protecting the system from unauthorized access and cyber-attacks originating from known malicious hosts.

4- Allowing Incoming HTTP and HTTPS Requests

```
[12/04/25]seed@VM:~$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
[12/04/25]seed@VM:~$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
[12/04/25]seed@VM:~$ sudo iptables -L -v
```

```

Chain INPUT (policy DROP 0 packets, 0 bytes)
  pkts bytes target     prot opt in     out     source
      destination
    0      0 DROP      all  --  any    any     10.0.1.10
      anywhere
    0      0 ACCEPT    tcp  --  any    any     anywhere
      anywhere          tcp dpt:http
    0      0 ACCEPT    tcp  --  any    any     anywhere
      anywhere          tcp dpt:https

Chain FORWARD (policy DROP 0 packets, 0 bytes)
  pkts bytes target     prot opt in     out     source
      destination
    0      0 ACCEPT    tcp  --  any    any     anywhere
      192.168.23.10      tcp dpt:http
    0      0 ACCEPT    tcp  --  any    any     anywhere
      192.168.23.10      tcp dpt:https

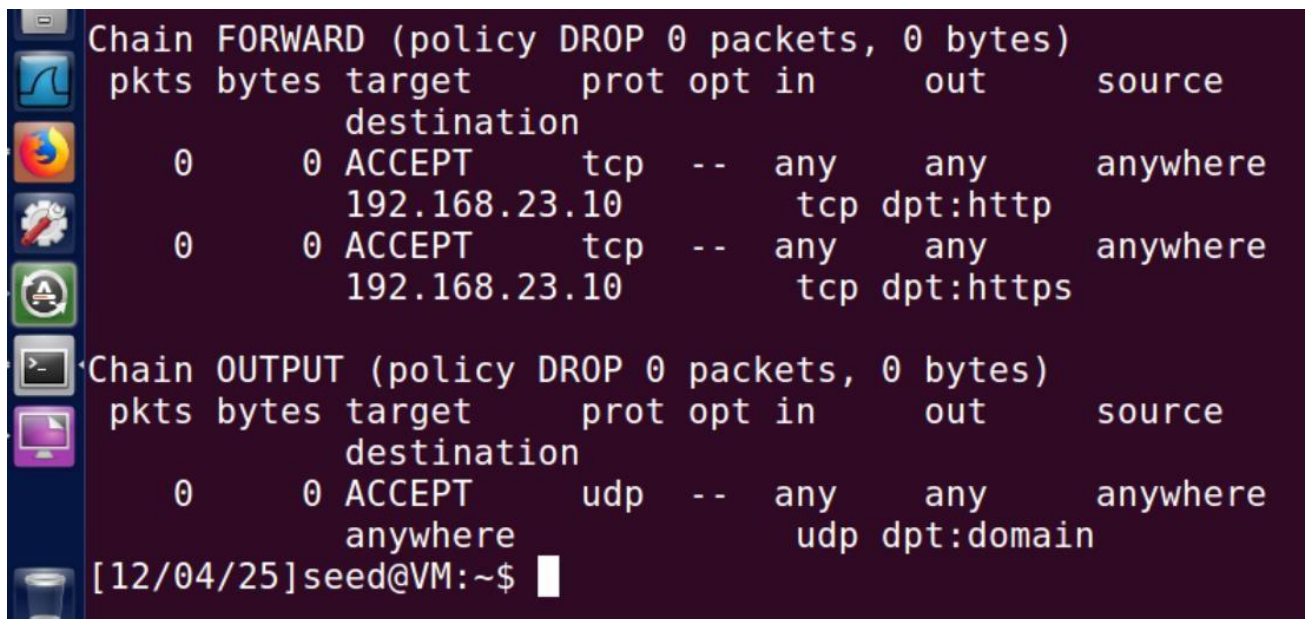
Chain OUTPUT (policy DROP 0 packets, 0 bytes)
  pkts bytes target     prot opt in     out     source
      destination
[12/04/25]seed@VM:~$

```

In this step, we added firewall rules to allow incoming TCP traffic on port **80 (HTTP)** and port **443 (HTTPS)** using `iptables`. Since the default policy is set to DROP, this ensures that web browsing traffic is not blocked and the client can securely access websites. The final command verifies that the rules were successfully applied.

5- Enabling Outbound DNS Traffic

```
[12/04/25]seed@VM:~$ sudo iptables -A OUTPUT -p udp --dport 53 -j ACCEPT
[12/04/25]seed@VM:~$ sudo iptables -L -v
Chain INPUT (policy DROP 0 packets, 0 bytes)
  pkts bytes target     prot opt in     out     source
      0      0 DROP      all  --  any    any     10.0.1.10
      0      0 anywhere
      0      0 ACCEPT    tcp  --  any    any     anywhere
      0      0 anywhere  tcp  --  any    any     anywhere
      0      0 ACCEPT    tcp  --  any    any     anywhere
      0      0 anywhere  tcp  --  any    any     anywhere
```



```
Chain FORWARD (policy DROP 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source
      destination
    0    0 ACCEPT      tcp  --  any     any     anywhere
      192.168.23.10      tcp dpt:http
    0    0 ACCEPT      tcp  --  any     any     anywhere
      192.168.23.10      tcp dpt:https

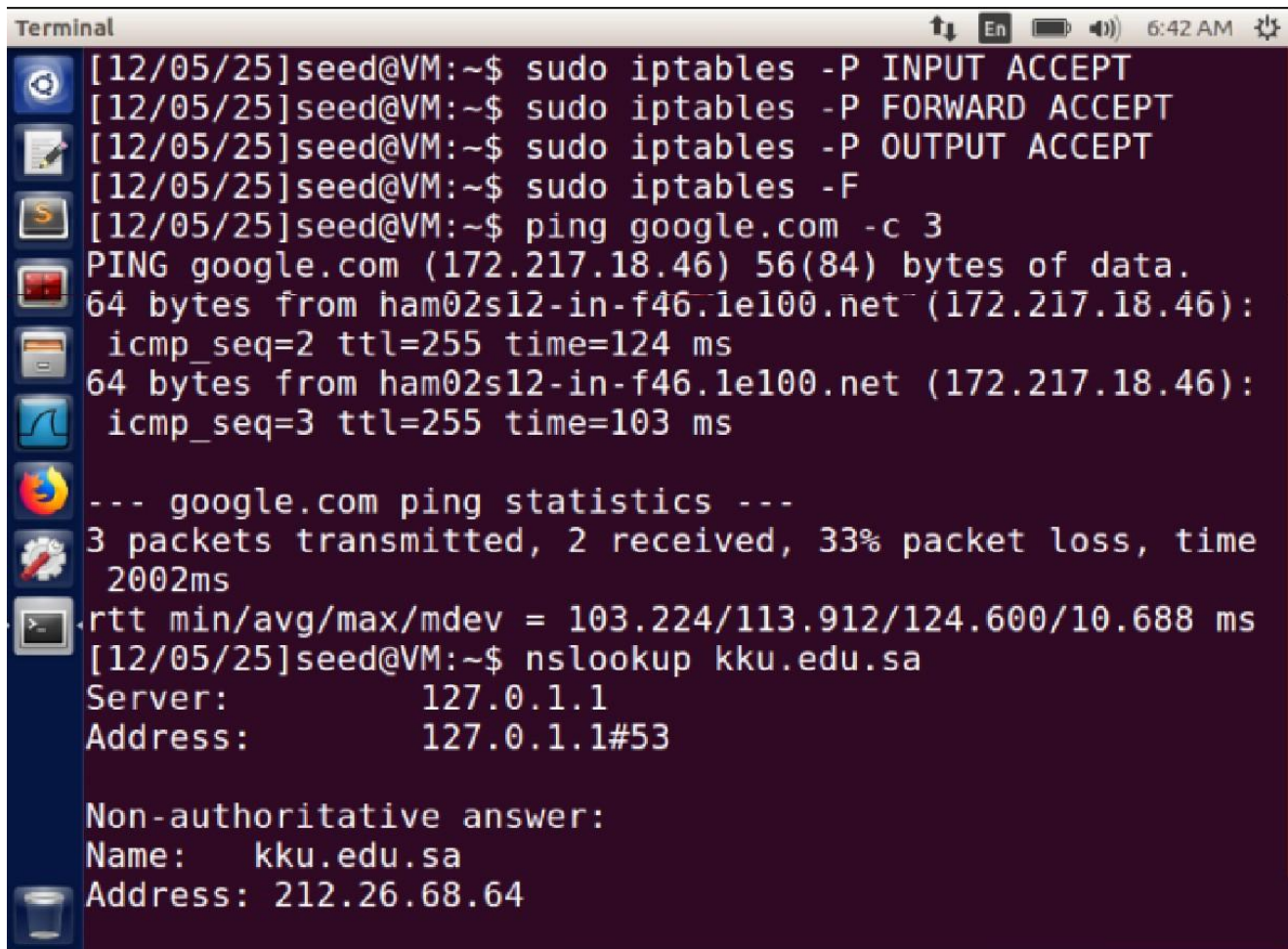
Chain OUTPUT (policy DROP 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source
      destination
    0    0 ACCEPT      udp  --  any     any     anywhere
      anywhere          udp dpt:domain

[12/04/25]seed@VM:~$
```

This rule allows the client to send DNS requests through port 53 so websites can be accessed normally. Since the default policy is DROP, we need to allow DNS traffic to keep internet browsing working while still blocking unwanted traffic.

Task-3(Establish a handshake session with any trusted website and show the following):

1. What is the IP address of the website >> use nslookup in the command terminal.

A screenshot of a Linux terminal window titled "Terminal". The window has a dark purple background and a light blue sidebar on the left with various application icons. The terminal shows the following commands and output:

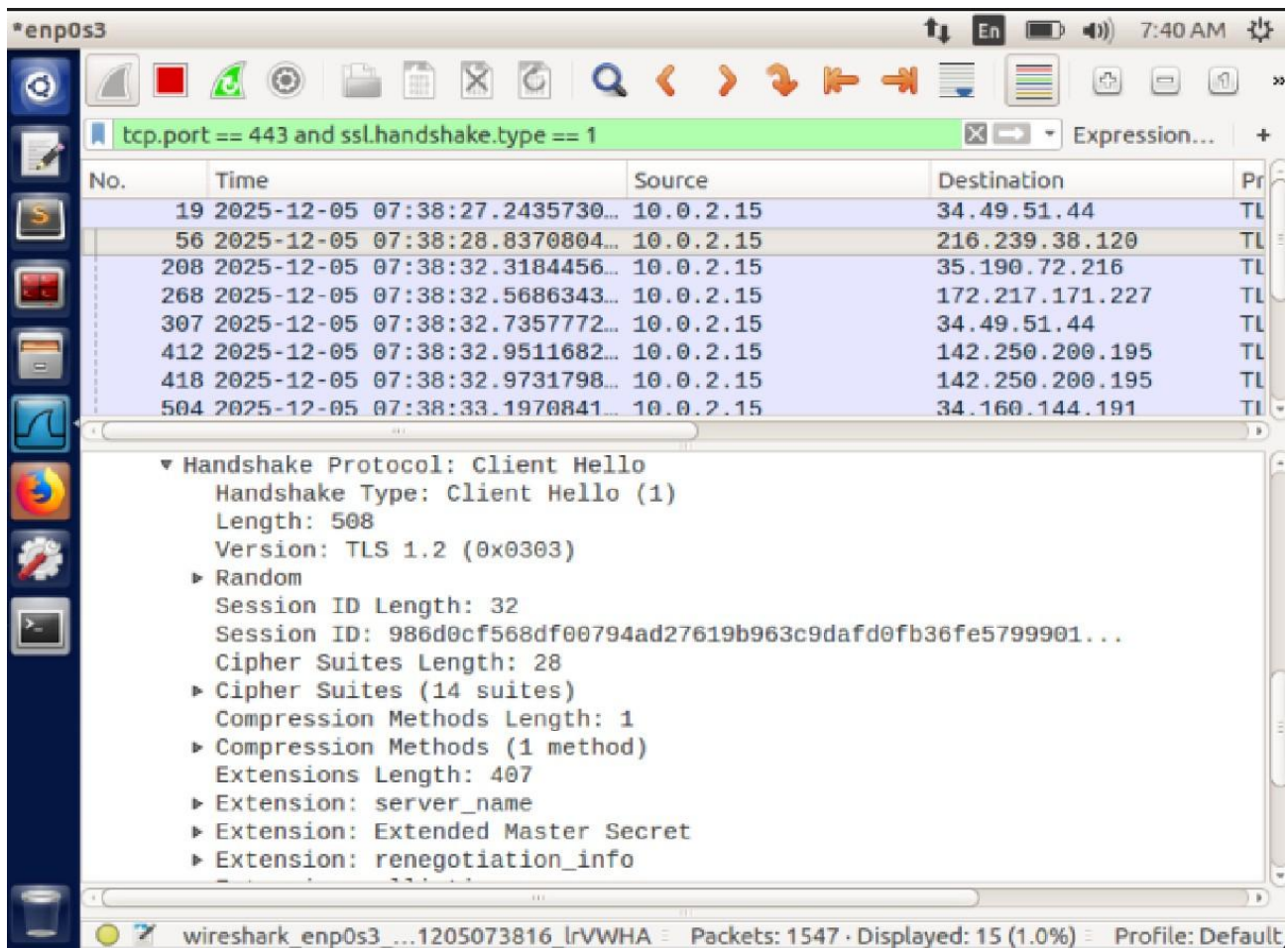
```
[12/05/25]seed@VM:~$ sudo iptables -P INPUT ACCEPT
[12/05/25]seed@VM:~$ sudo iptables -P FORWARD ACCEPT
[12/05/25]seed@VM:~$ sudo iptables -P OUTPUT ACCEPT
[12/05/25]seed@VM:~$ sudo iptables -F
[12/05/25]seed@VM:~$ ping google.com -c 3
PING google.com (172.217.18.46) 56(84) bytes of data.
64 bytes from ham02s12-in-f46.1e100.net (172.217.18.46):
  icmp_seq=2 ttl=255 time=124 ms
64 bytes from ham02s12-in-f46.1e100.net (172.217.18.46):
  icmp_seq=3 ttl=255 time=103 ms

--- google.com ping statistics ---
3 packets transmitted, 2 received, 33% packet loss, time
 2002ms
rtt min/avg/max/mdev = 103.224/113.912/124.600/10.688 ms
[12/05/25]seed@VM:~$ nslookup kku.edu.sa
Server:          127.0.1.1
Address:         127.0.1.1#53

Non-authoritative answer:
Name:   kku.edu.sa
Address: 212.26.68.64
```

This step uses the `nslookup` command to obtain the IP address of the KKU website. The IP address will help us identify the packets related to KKU later in Wireshark.

2- Client Hello message and show a cipher suite supported by your browser.



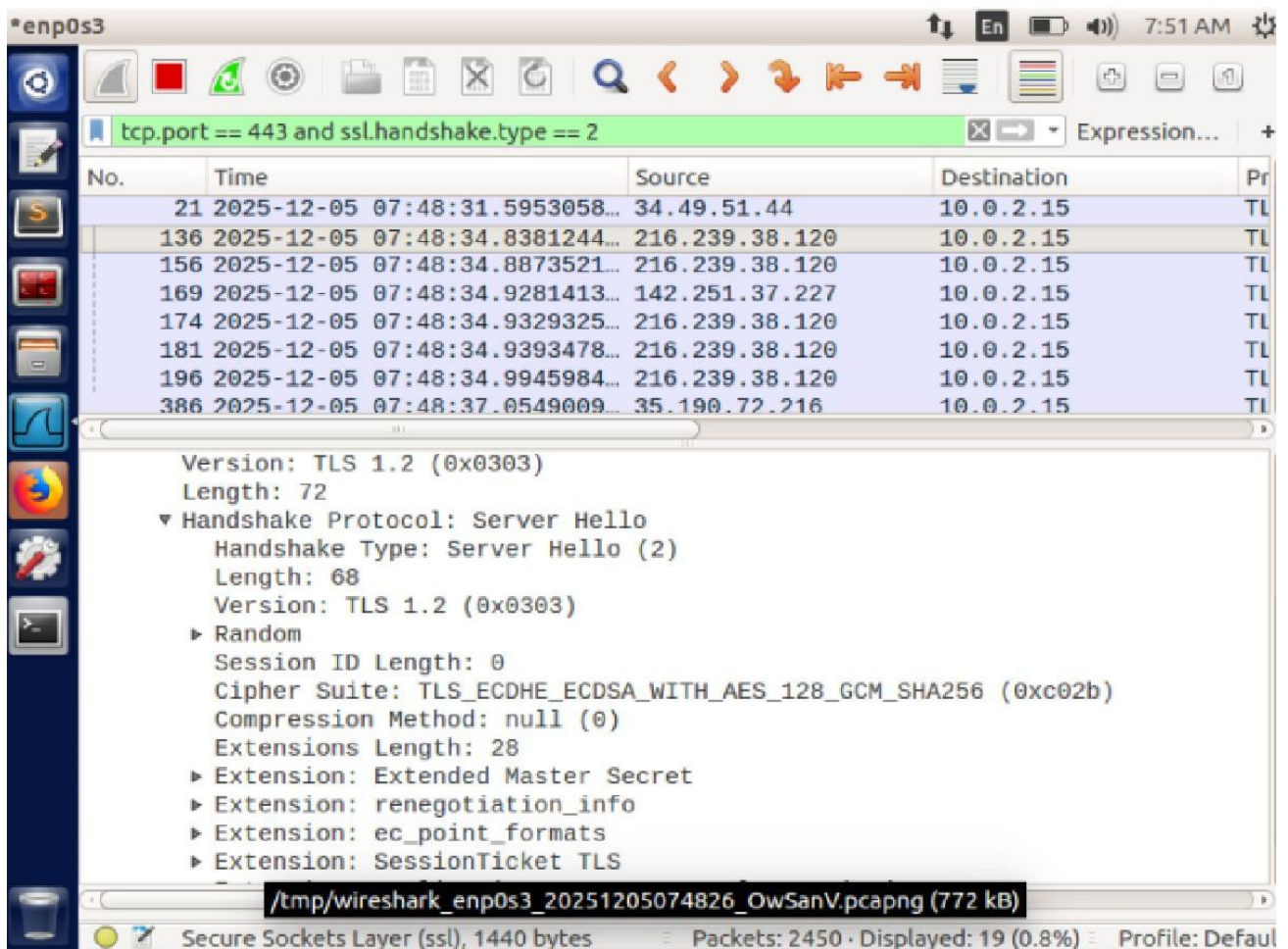
When establishing an HTTPS secure connection, the client sends the first message called Client Hello to the server as part of the TLS handshake.

This packet includes:

- TLS Version supported by the browser → (TLS 1.2 as shown in the capture).
- Cipher Suites list which are the encryption algorithms that the browser can use with the server.
- Session ID and random value used for generating encryption keys.
- Additional extensions such as *Server Name Indication (SNI)*.

By capturing this packet in Wireshark, we can identify what encryption methods the browser supports to secure communication.

3- What is the compression method?



The image shows a Wireshark packet capture window titled '*enp0s3'. The filter bar at the top is set to 'tcp.port == 443 and ssl.handshake.type == 2'. The packet list shows several packets, with packet 21 selected. The packet details pane shows the following information:

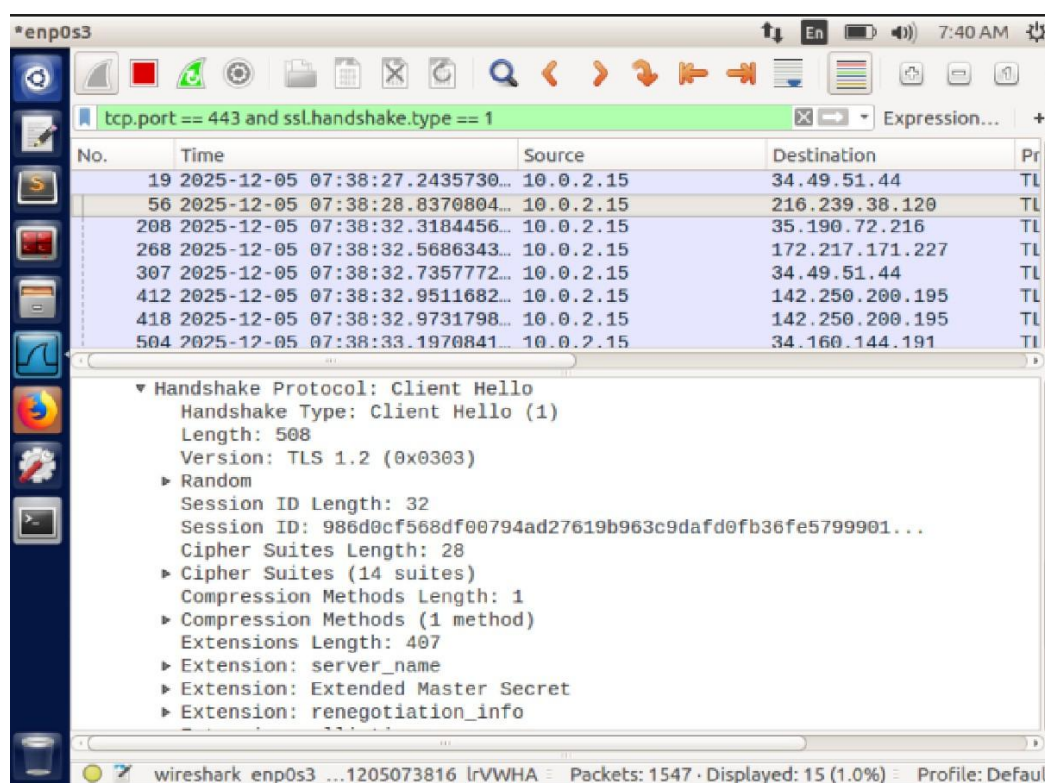
- Version: TLS 1.2 (0x0303)
- Length: 72
- Handshake Protocol: Server Hello
 - Handshake Type: Server Hello (2)
 - Length: 68
 - Version: TLS 1.2 (0x0303)
 - Random
 - Session ID Length: 0
 - Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b)
 - Compression Method: null (0)
 - Extensions Length: 28
 - Extension: Extended Master Secret
 - Extension: renegotiation_info
 - Extension: ec_point_formats
 - Extension: SessionTicket TLS

The status bar at the bottom indicates 'Secure Sockets Layer (ssl), 1440 bytes', 'Packets: 2450', 'Displayed: 19 (0.8%)', and 'Profile: Default'.

In the TLS Server Hello message, the compression method used by the server is "null (0)". This means that no compression is applied to the TLS records during transmission. Modern secure servers disable TLS compression to protect against security vulnerabilities such as the CRIME attack, which can exploit compressed data to leak sensitive information.

Therefore, using "null" compression is the secure and recommended standard in current HTTPS communications.

4- What is the session ID?



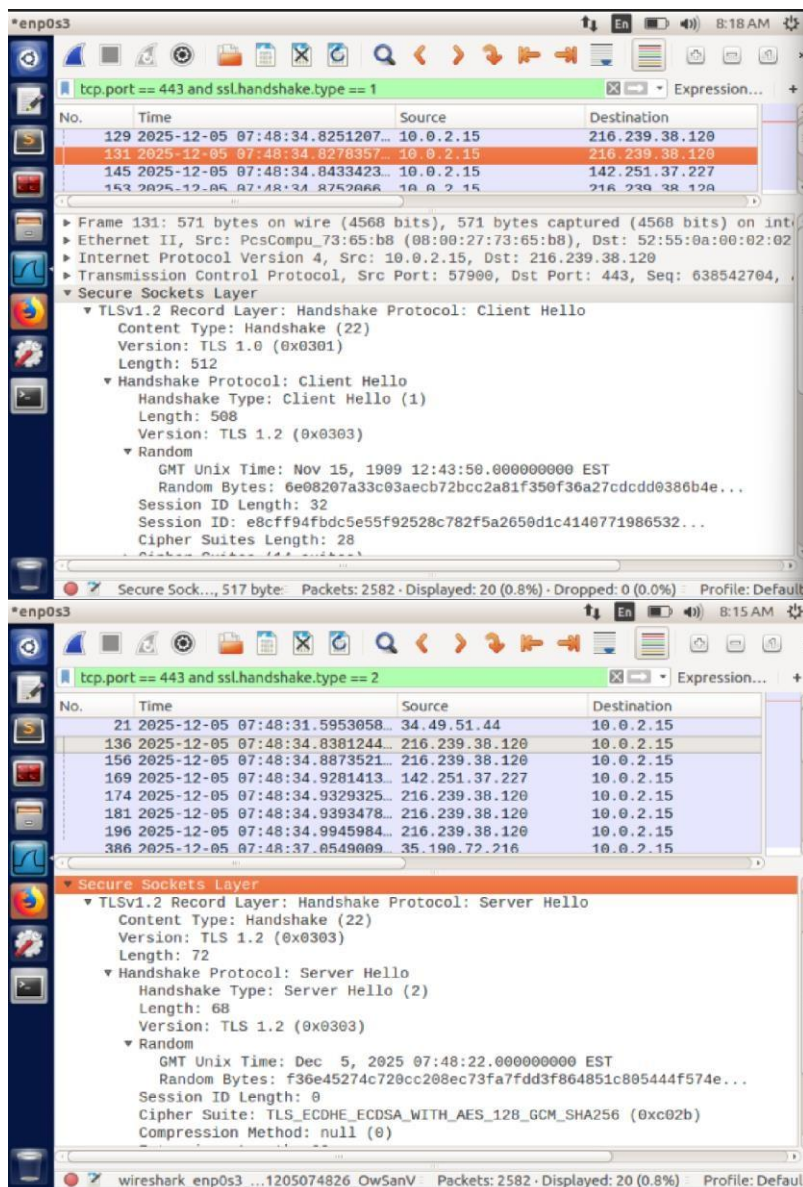
During the TLS handshake process, the client includes a Session ID within the *Client Hello* message. The Session ID serves as a unique identifier that allows the client and server to resume a previously established secure session without performing the full handshake again. This mechanism enhances performance by reducing latency and minimizing cryptographic overhead.

In our Wireshark capture of the Client Hello packet, the Session ID details were as follows:

- Session ID Length: 32 bytes
- Session ID Value:
98d60cf568df0079ad27619b963c9dadf0fb36fe5799901...

The presence of this Session ID indicates that the client is either attempting to resume an earlier TLS session or allowing the server to resume the session if supported.

5- What is a random number?



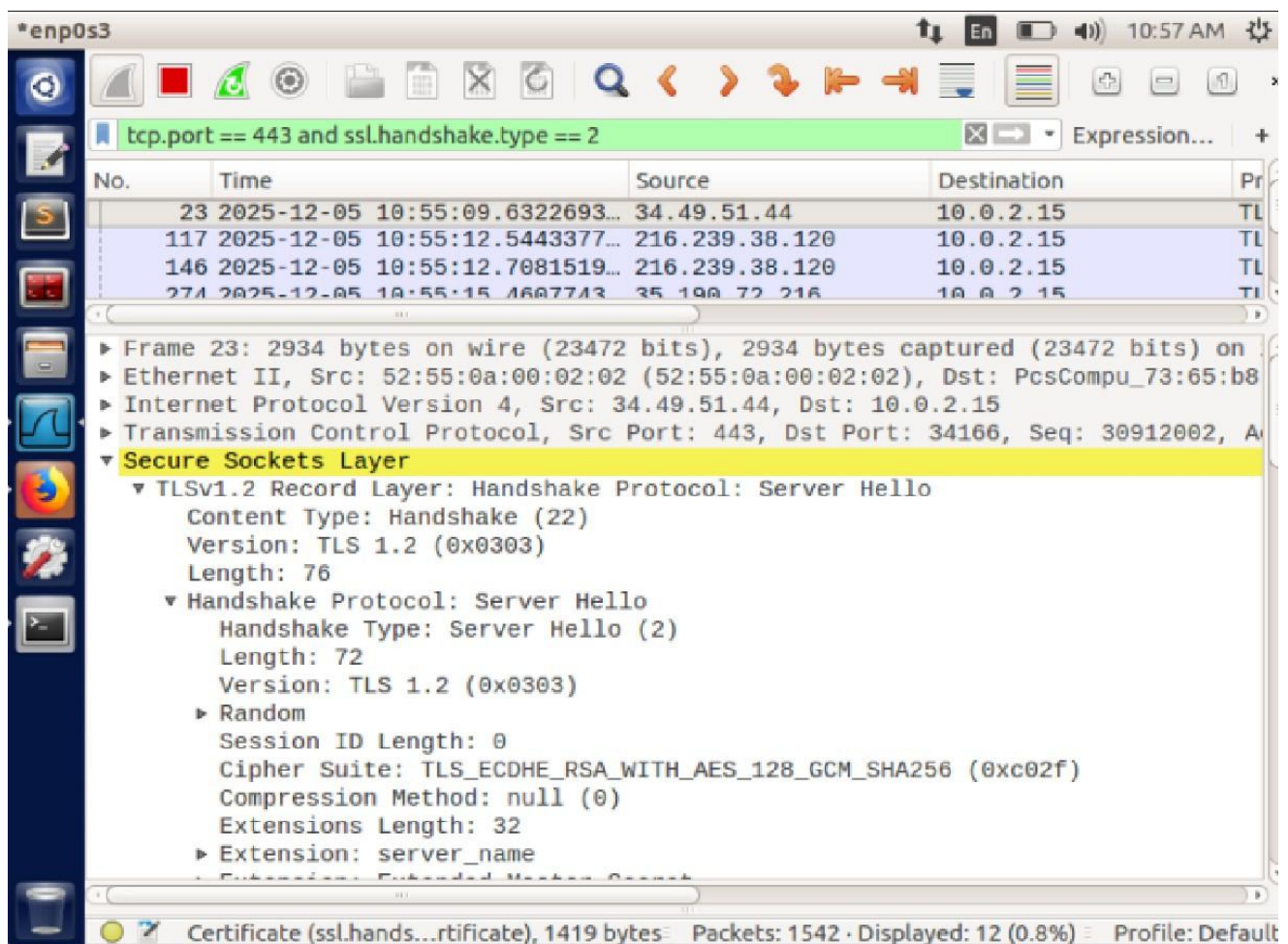
In the Client Hello message, the TLS protocol includes a Random value which consists of:

- A 4-byte Unix timestamp, and
- 28 bytes of cryptographically strong random data

This random number plays a crucial role in the key generation process, as it helps both the client and server generate fresh session keys during each TLS handshake.

By using random values, the TLS protocol protects against replay attacks and ensures the uniqueness and security of every encrypted session.

6- Inspect Server Hello, what did the server choose in the cipher suite?



In the Server Hello message, the server selects one cipher suite from the list offered by the client to secure the TLS session. The selected cipher suite in our captured packet is:

`TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256`

This cipher suite provides strong encryption and authentication using ECDHE and AES-GCM, helping ensure the confidentiality and integrity of the connection.

7- Is the session ID the same from Server Hello What does this refer to ?

The Session ID in the Server Hello is not the same as the one in the Client Hello. This indicates that the server did not resume a previous TLS session and instead created a fresh session.

A different Session ID means the server requires a full TLS handshake to establish new encryption keys.

8- Is the random number the same from Server Hello? Why ?

During the TLS handshake, the random number in the Server Hello message is not the same. The reason is that both the client and the server contribute random values to the key generation process. As a result, the encryption keys for each session are unique and unpredictable, even in the case where the same client and server reconnect.

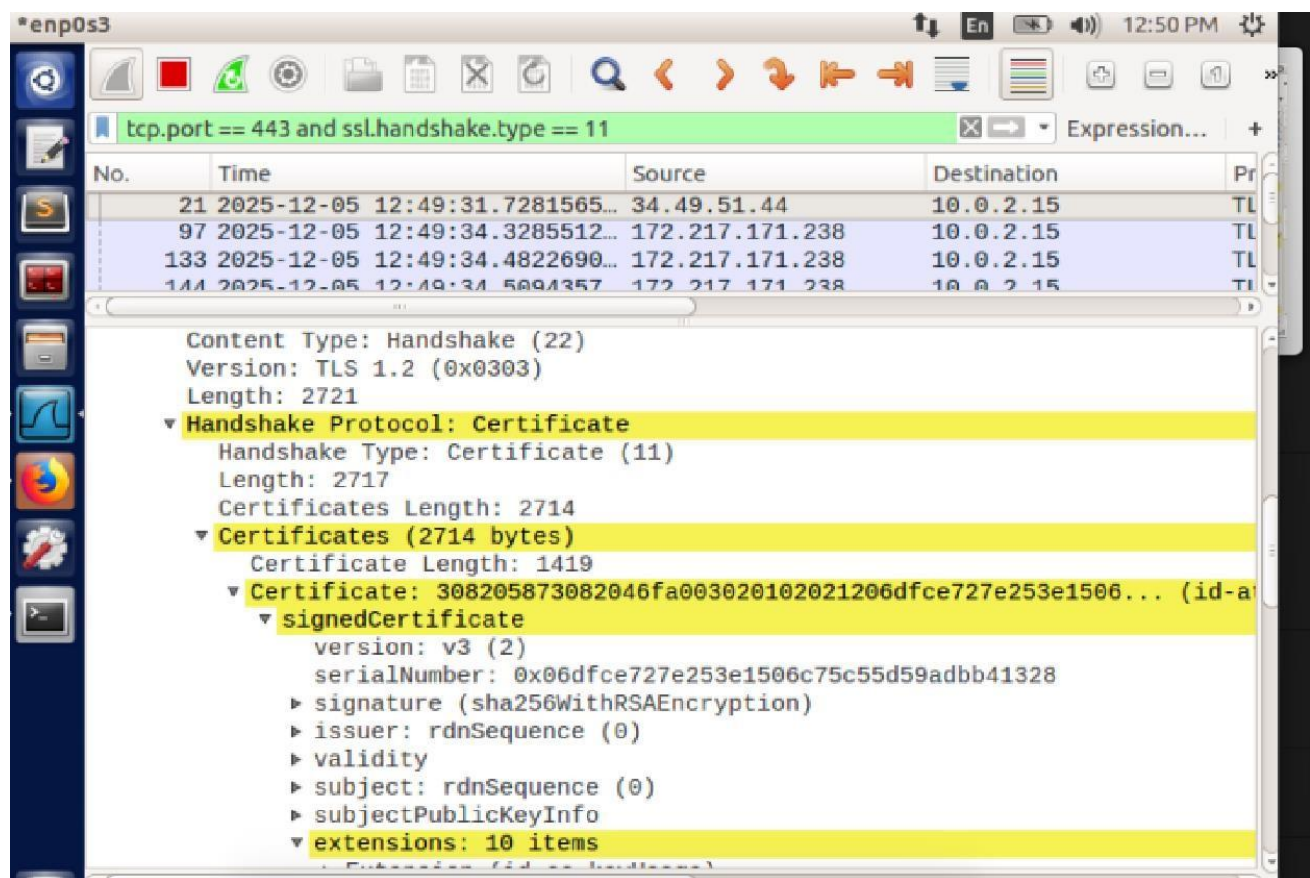
9- Is there a certificate packet between KCU and your browser? Why ?

Yes, there is a certificate packet. The server must present its digital certificate to the client during the TLS handshake as a method of authentication. The client's browser then validates this certificate using its list of trusted Certificate Authorities (CAs) to ensure the server's authenticity and establish a secure, encrypted connection.

10- Inspect Change cipher spec, do you see the contents of the message? If not, why ?

In the Change Cipher Spec packet, the contents of the message are encrypted. This is because the TLS protocol switches to encrypted communication after negotiating the security parameters during the handshake. As a result, Wireshark cannot decode the application data, ensuring confidentiality and preventing eavesdropping.

11- Open any other website (https) and show the certificate of the website and explain its contents captured by Wireshark.



The Wireshark capture displays the TLS Certificate message that the web server sends as part of its authentication process during the handshake. In this step, the server provides its own certificate as well as the certificates of the intermediate CA to form a trusted certificate chain. The certificate includes essential information such as the website's identity (Subject), the Certificate Authority that issued it (Issuer), the certificate validity period, and the server's public key. Using this information, the client verifies the authenticity of the server and ensures that a secure encrypted connection can be established. This was clearly observed in our Wireshark capture, where the certificate details of the visited HTTPS website were extracted and analyzed.